

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Аппаратное и программное обеспечение сетей»

Тема: «Программирование сетевых приложений на базе платформы java и библиотеки
java.net.»

Выполнил:

Студент 3 курса

Группы ИИ-28

Асташёнок Д.С.

Проверила:

Кулик А.Д.

Брест 2026

Цель работы: Цель цикла работ – Изучить основы программирования сетевых приложений на базе библиотеки java.net. Приобрести навыки по практическому использованию библиотеки для реализации сетевых приложений на базе протоколов TCP.

Вариант 2.7

Ход работы:

TcpClient.java:

```
import java.awt.BorderLayout;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;
import java.io.*;
import java.net.*;
import java.util.Date;
import javax.swing.*;

public class TcpClient extends JFrame {
    private static final String DEFAULT_HOST = "127.0.0.1";
    private static final int DEFAULT_PORT = 8080;

    private Socket socket;
    private PrintWriter out;
    private String connectedHost;
    private int connectedPort;
    private boolean startedLogged;
    private final JTextArea log = new JTextArea();
    private final JTextField input = new JTextField();
    private final PrintWriter journal;

    public TcpClient() throws IOException {
        super("TCP client, var 2 / client 7");
        journal = new PrintWriter(new FileWriter("tcp-client.log",
true), true);
        log.setEditable(false);
        add(new JScrollPane(log), BorderLayout.CENTER);

        JButton sendBtn = new JButton("Отправить");
        sendBtn.addActionListener(e -> sendLine());
        input.addActionListener(e -> sendLine());
        input.addKeyListener(new KeyAdapter() {
            public void keyPressed(KeyEvent e) {

                if (e.getKeyCode() == KeyEvent.VK_PAGE_DOWN)
sendLine();
            }
        });
        JPanel south = new JPanel(new BorderLayout(4, 0));
        south.add(input, BorderLayout.CENTER);
        south.add(sendBtn, BorderLayout.EAST);
        add(south, BorderLayout.SOUTH);
    }
}
```

```

        setSize(560, 400);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        addWindowListener(new java.awt.event.WindowAdapter() {
            public void windowClosing(java.awt.event.WindowEvent e)
        {
            closeConn();
        }
    });
    log.append("Автоподключение к " + DEFAULT_HOST + ":" +
DEFAULT_PORT + "\n");
    log.append("Отправка: кнопка «Отправить», PgDn
( Fn+Down )\n");
    log.append("Разрыв: disconnect " + DEFAULT_HOST + " " +
DEFAULT_PORT + "\n");
    setVisible(true);
    tryConnect();
    input.requestFocusInWindow();
}

private synchronized boolean tryConnect() {
    if (out != null) return true;
    try {
        socket = new Socket(DEFAULT_HOST, DEFAULT_PORT);
        out = new PrintWriter(new BufferedWriter(
            new
OutputStreamWriter(socket.getOutputStream()), true);
        connectedHost = DEFAULT_HOST;
        connectedPort = DEFAULT_PORT;
        startedLogged = true;
        journal.println(new Date() + " CONNECTION START " +
connectedHost + ":" + connectedPort);
        log.append("подключен к " + connectedHost + ":" +
connectedPort + "\n");
        new Thread(this::readLoop).start();
        return true;
    } catch (Exception ex) {
        log.append("нет сервера: " + ex.getMessage() + "
(запусти TcpServer и отправь ещё раз)\n");
        socket = null;
        out = null;
        return false;
    }
}

private void sendLine() {
    String text = input.getText();
    input.setText("");
    if (text.isEmpty()) return;

    if (text.startsWith("disconnect")) {
        String[] hp = parseHostPort(text, "disconnect");
        if (hp == null) {
            log.append("Формат: disconnect <адрес> <порт>\n");
            return;
        }
    }
}

```

```

        synchronized (this) {
            if (out == null) {
                log.append("соединения нет\n");
                return;
            }
            if (!hp[0].equals(connectedHost) ||
Integer.parseInt(hp[1]) != connectedPort) {
                log.append("не совпадает с текущим " +
connectedHost + ":" + connectedPort + "\n");
                return;
            }
        }
        log.append("disconnect " + hp[0] + ":" + hp[1] + "\n");
        closeConn();
        return;
    }

    if (text.startsWith("connect ")) {
        log.append("автоподключение, команда connect не
нужна\n");
        return;
    }

    synchronized (this) {
        if (out == null && !tryConnect()) return;
        out.println(text);
        journal.println(new Date() + " SENT " + text);
    }
    log.append(">>> " + text + "\n");
}

private static String[] parseHostPort(String text, String cmd) {
    String rest = text.substring(cmd.length()).trim();
    if (rest.isEmpty()) return null;
    rest = rest.replace(':', ' ');
    String[] p = rest.split("\\s+");
    if (p.length != 2) return null;
    try {
        Integer.parseInt(p[1]);
    } catch (NumberFormatException e) {
        return null;
    }
    return p;
}

private void readLoop() {
    try {
        BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        String line;
        while ((line = in.readLine()) != null) {
            String msg = line;
            SwingUtilities.invokeLater(() -> log.append("<<< " +
msg + "\n"));
        }
    } catch (IOException e) {

```

```

    }
    SwingUtilities.invokeLater(() -> log.append("соединение
закрыто\n"));
    closeConn();
}

private synchronized void closeConn() {
    if (socket == null) return;
    if (startedLogged) {
        journal.println(new Date() + " CONNECTION END");
        startedLogged = false;
    }
    try { socket.close(); } catch (IOException ignored) {}
    socket = null;
    out = null;
    connectedHost = null;
    connectedPort = 0;
}

public static void main(String[] args) throws IOException {
    new TcpClient();
}
}

```

TcpServer.java:

```

import java.io.*;
import java.net.*;

class ServeOne extends Thread {
    private Socket socket;
    private BufferedReader in;
    private PrintWriter out;

    public ServeOne(Socket s) throws IOException {
        socket = s;
        in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        out = new PrintWriter(new BufferedWriter(
            new OutputStreamWriter(socket.getOutputStream())),
true);
        start();
    }

    public void run() {
        try {
            out.println("Hello, Student!");
            String line;
            while ((line = in.readLine()) != null) {
                if (line.startsWith("save ")) {
                    String name = line.substring(5).trim();

```

```

        StringBuilder content = new StringBuilder();
        String part;
        boolean ok = false;
        while ((part = in.readLine()) != null) {
            int p = part.indexOf("#end##");
            if (p >= 0) {
                content.append(part.substring(0, p));
                ok = true;
                break;
            }
            content.append(part).append('\n');
        }
        if (!ok) break;
        try {
            FileWriter fw = new FileWriter(name);
            fw.write(content.toString());
            fw.close();
            out.println("OK: file " + name + "
created");
        } catch (IOException e) {
            out.println("ERROR: cannot create file");
            break;
        }
    } else {
        out.println(line);
    }
}
} catch (IOException e) {
    System.err.println("IO Exception");
} finally {
    try {
        socket.close();
    } catch (IOException e) {
        System.err.println("Socket not closed");
    }
}
}
}

```

```

public class TcpServer {
    static final int PORT = 8080;

    public static void main(String[] args) throws IOException {
        ServerSocket s = new ServerSocket(PORT);
        System.out.println("TCP server started on " + PORT);
        try {
            while (true) {
                Socket socket = s.accept();
                try {
                    new ServeOne(socket);
                } catch (IOException e) {
                    socket.close();
                }
            }
        } finally {
            s.close();
        }
    }
}

```

```
    }  
    }  
}
```

Вывод: Изучил основы программирования сетевых приложений на базе библиотеки java.net; Приобрёл навыки по практическому использованию библиотеки для реализации сетевых приложений на базе протоколов TCP